PROGRAM:

```
#include <stdio.h>
#include <malloc.h>

struct node
{
    int data;
    struct node *prev, *next;
};

struct node *head, *newnode, *temp, *tail, *before, *curr;
int total_elements = 0;

void delete(int data)
{
    int found = 0;
    before = NULL;
    curr = head;
    while(curr != NULL)
    {
        if(curr->data == data)
        {
            found = 1;
            break;
        }
        before = curr;
        curr = curr->next;
    }
    if(found == 0)
    {
        printf("\nelement not found");
    }
    else
    {
        if(curr == head)
        {
            head = curr->next;
            if(head != NULL)
                head->prev = NULL;
            free(curr);
```

```
        }
        else if(curr->next == NULL)
        {
            before->next = NULL;
            tail = before;
            free(curr);
        }
        else
        {
            before->next = curr->next;
            curr->next->prev = before;
            free(curr);
        }
        total_elements--;
    }
}

void insert(int data, int pos)
{
    newnode = (struct node*)malloc(sizeof(struct node));
    newnode->data = data;
    newnode->prev = NULL;
    newnode->next = NULL;

    if(pos < 1 || pos > total_elements + 1)
    {
        printf("\nInvalid position");
        return;
    }

    if(pos == 1)
    {
        if(head == NULL)
        {
            head = tail = newnode;
        }
        else
        {
            newnode->next = head;
            head->prev = newnode;
            head = newnode;
        }
```

```
    }
    else if(pos == total_elements + 1)
    {
        tail->next = newnode;
        newnode->prev = tail;
        tail = newnode;
    }
    else
    {
        temp = head;
        for(int i = 1; i < pos - 1; i++)
        {
            temp = temp->next;
        }
        newnode->next = temp->next;
        newnode->prev = temp;
        temp->next->prev = newnode;
        temp->next = newnode;
    }
    total_elements++;
}

void display()
{
    temp = head;
    if(temp == NULL)
    {
        printf("\nList is empty\n");
        return;
    }
    printf("\nCurrent List: ");
    while(temp != NULL)
    {
        printf("%d", temp->data);
        if(temp->next != NULL) printf(" <-> ");
        temp = temp->next;
    }
    printf("\n");
}

int main()
{
```

```
    printf("*** DOUBLY LINKED LIST OPERATIONS ***\n");

    insert(10, 1);
    insert(20, 2);
    insert(30, 3);
    display();

    printf("\nDeleting element 20...\n");
    delete(20);
    display();

    printf("\nDeleting element 50...\n");
    delete(50);
    printf("\n");

    return 0;
}
```

OUTPUT:

```
*** DOUBLY LINKED LIST OPERATIONS ***

Current List: 10 <-> 20 <-> 30

Deleting element 20...

Current List: 10 <-> 30

Deleting element 50...
element not found
```